

THEORETICAL ASSIGNMENT 12

PySpark • Delta Lake • Slowly Changing Dimensions (SCD)

1. Upsert (Update + Insert) in Delta Lake

Upsert checks whether a source record exists in the target. Existing records are updated; missing records are inserted. Delta Lake uses `merge()` with `whenMatchedUpdate()` and `whenNotMatchedInsert()`.

```
upsert_data = [(9, 'kiran'), (8, 'manoj'), (7, 'vikas')]
source_df = spark.createDataFrame(upsert_data, ['id', 'name'])

deltaTable.alias('target') \
    .merge(source=source_df.alias('source'),
           condition='target.id = source.id') \
    .whenMatchedUpdate(set={'name': 'source.name'}) \
    .whenNotMatchedInsert(values={
        'id': 'source.id', 'name': 'source.name'
    }).execute()
```

How it works: The aliases identify target and source. The ID condition matches records. A match triggers the update clause; no match triggers the insert clause. `execute()` runs the operation.

Source record	Action	Reason
(9, 'kiran')	Insert	ID 9 absent
(8, 'manoj')	Update	ID 8 exists
(7, 'vikas')	Update	ID 7 exists

2. What is SCD? How many types?

SCD means **Slowly Changing Dimensions**. It describes methods for managing changes in dimension data such as customers, products, or employees over time.

Date	Mobile number
2020-01-01	1234567890
2024-08-01	1234567896
2026-09-19	1223454235

Type	Approach
Type 0	Retain original value
Type 1	Overwrite; no history in current row
Type 2	Add a new row for each update
Type 3	Add a column for limited history
Type 4	Separate history table
Type 6	Hybrid: Types 1 + 2 + 3

3. SCD types with examples

Type 0 — Fixed / Retain Original

Retains the original value even if a new value arrives. Class examples: Aadhaar number or PAN card number. The stored original is not changed.

Customer	Aadhaar number
Customer 1	1234567890

Type 1 — Overwrite (No History)

Overwrites the existing value with the latest value. Earlier values are not retained in the current dimension row. In the mobile example, only 1223454235 remains.

```
deltaTable.update(
  condition='id = 7',
  set={'name': "'sai'"}
)
```

This updates ID 7 name to sai. The prior name is not retained in that row.

Type 2 — New Row for Every Update

Adds a new row when a tracked attribute changes. The prior row remains for history. The class table uses is_current, start_date, and end_date.

id	name	is_current	start_date	end_date
9	kiran	False	2020-01-01	2024-01-01
9	Kumar	False	2024-01-01	2026-09-19
9	varun	True	2026-09-19	NULL

Meaning: is_current identifies the current version. start_date is when the version starts; end_date is when it ends. The current row has True and a NULL end date.

Class PySpark flow:

```
source_data = [(9, 'vikram')]
source_df = spark.createDataFrame(source_data, ['id', 'name'])
target_df = deltaTable.toDF()

df_updates = source_df.join(target_df, 'id') \
    .filter('is_current = true') \
    .select(source_df.id, source_df.name)

staged_update = source_df.selectExpr('id as merge_key', '**') \
    .unionByName(df_updates.selectExpr('NULL as merge_key', '**'))

current_date_str = datetime.datetime.now().strftime('%Y-%m-%d')

deltaTable.alias('target') \
    .merge(source=staged_update.alias('source'),
          condition='target.id = source.merge_key and target.is_current = True') \
    .whenMatchedUpdate(
        condition='target.name != source.name',
        set={'end_date': f"'{current_date_str}'", 'is_current': 'False'}
    ) \
    .whenNotMatchedInsert(values={
        'id': 'source.id', 'name': 'source.name', 'dept': 'NULL',
        'start_date': f"'{current_date_str}'", 'end_date': 'NULL',
        'is_current': 'True'
    }).execute()
```

The staged row with merge_key = id matches and expires the old current row by setting its end date and is_current=False. A second staged row has NULL merge_key, matches nothing, and is inserted as the new current version with the current start date.

Type 3 — New Column (Limited History)

Adds a column to retain a previous value, commonly keeping current and previous values:

current_mobile_no	previous_mobile_no
1223454235	1234567896

On a change, the current value is updated and the previous value is moved into the previous column. History is limited.

Type 4 — Separate History Table

The main table stores only latest information; a separate history table stores previous values.

Main table: Customer	Mobile number
Customer 1	1223454235

History table: Customer	Mobile number
Customer 1	1234567890
Customer 1	1234567896

Type 6 — Hybrid (Types 1 + 2 + 3)

Combines Type 1 overwriting, Type 2 historical rows, and Type 3 additional column(s) for limited history. A design may retain historical rows while also storing current and previous values in columns.

Customer	Current mobile	Previous mobile	is_current
Customer 1	1223454235	1234567896	True

Quick revision

Type	Handling	History
0	Keep original	Original only
1	Overwrite	No history in dimension row
2	Insert new row	Full row-level history
3	Add column	Limited
4	Separate history table	Stored separately
6	Types 1 + 2 + 3	Hybrid

Type 1 vs Type 2: Type 1 updates the existing row and keeps the latest value. Type 2 expires the old row and inserts a new version, preserving history.

Final takeaway: SCD defines how dimension changes are stored. Type 1 replaces old values; Type 2 preserves versions as rows; Types 0, 3, 4, and 6 provide other ways to retain or manage history.